Advanced Data Science Project · Prototype validation · Team DSProject5DS1

OpsAgents: a guarded multi-agent assistant for DevOps and MLOps

Five specialist AI agents diagnose failed builds, review pull requests, untangle infrastructure, resolve merge conflicts and watch models for drift. An orchestrator coordinates them, and a guardrail makes sure no change reaches production without the right human approval.

Lyna · Dependency Resolver
zrharounesprit · Diagnostic / RCA
Muaad · Merge Assistant
hendch · Code Reviewer
omar · Drift Detection
Rayen · Guardrail, orchestrator, platform

1

Problem and targeted objectives

The problem

Teams that ship software and machine learning models lose hours to the same incidents: a CI build fails and someone has to dig through a long log; an infrastructure deploy fails on a dependency cycle; two branches conflict; a pull request quietly leaks test data into training; a model in production degrades because its input data changed. Each incident needs a specialist, and they tend to happen at the worst time.

AI agents can do much of this diagnosis. Letting them act on their own is risky, though: an agent that edits IAM permissions or rolls back a production model without oversight can cause more damage than the incident it was fixing.

Target users

- **Small DevOps and platform teams** without round-the-clock specialists.
- **Data science teams** that deploy models and own their pipelines.
- **Team leads and reviewers** who approve changes and need to know why each one was proposed.

Main objective

Cut the time from "something broke" to "we know why and have a fix", while every change that carries risk stays under human control and leaves an auditable trail.

Specific objectives

01 **Diagnose failed builds automatically**
A ranked list of root causes, with the evidence each one relies on, within a minute of a failed CI run.

02 **Review every pull request for ML mistakes**
Data leakage, unsafe preprocessing and lint issues, with a minimal fix proposed.

03 **Cover the rest of the delivery chain**
Infrastructure dependency cycles, merge conflicts and model drift, each handled by a dedicated agent.

04 **Never let an agent make a risky change alone**
Each proposal is risk-scored, checked against a written policy and held for one or two human approvals when needed.

05 **Make every decision traceable**
A tamper-evident audit log of decisions, plus LangSmith traces of every agent step and model call.

06 **Keep working when an AI model fails**
Every agent has a rule-based fallback, and the run is marked "degraded" rather than failing.

How the solution addresses it

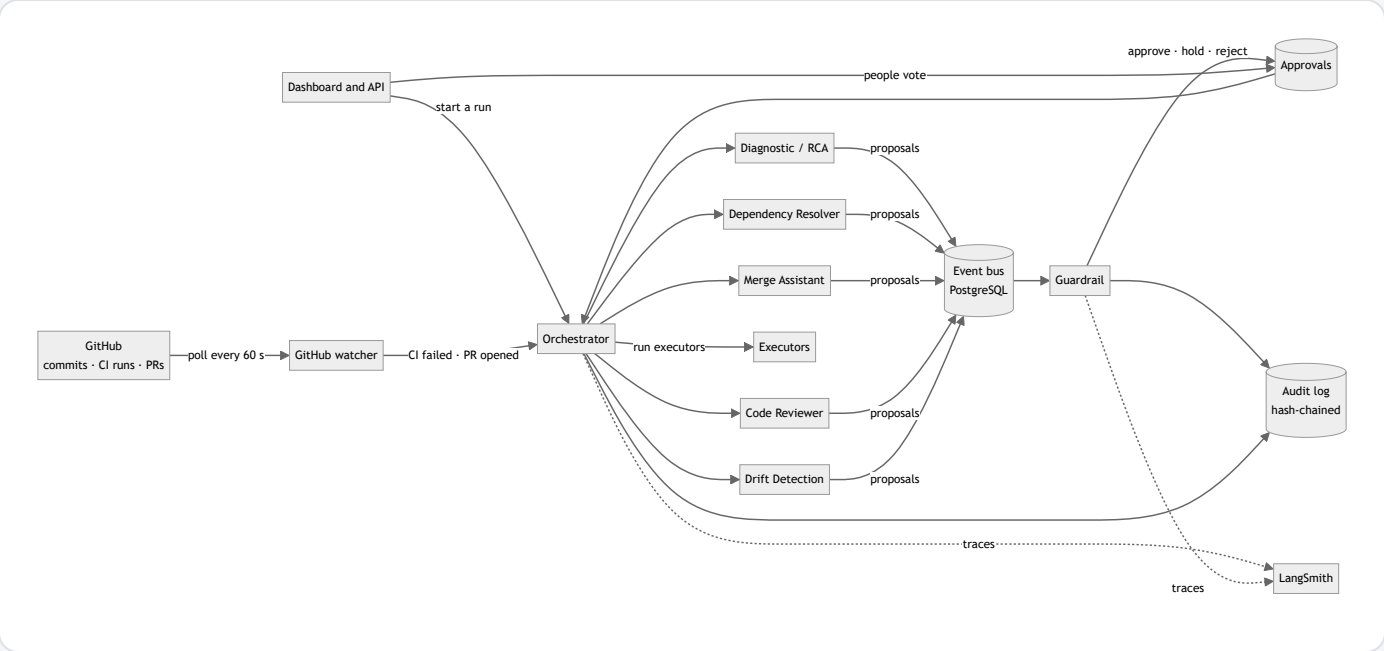
OpsAgents connects to the team's GitHub repositories. When CI fails or a pull request opens, the orchestrator sends the right specialists. Their findings arrive as *proposals*, never as direct actions. The Guardrail scores each proposal's real risk and applies the team's policy. Advice and low-risk changes go through at once. Risky changes wait in an

approval queue on the dashboard, where the person who triggered the run cannot approve it themselves. Everything is recorded.

2

Multi-agent architecture

The system is an **orchestrator–worker** design with a separate **supervisor** (the Guardrail). Worker agents never call each other. The orchestrator calls them in order, passes what earlier agents found to later ones, and publishes each proposal as an event on a PostgreSQL event bus. The Guardrail consumes those events and decides. All components share one PostgreSQL database and one Python environment.



The agents

For each agent: its objective, responsibilities, model and the reason for that model, and its tools.

Orchestrator

Rayen

Objective	Turn each trigger into a complete, recoverable run: pick the agents, run them, collect decisions, execute and summarise.
Responsibilities	Routing (rules first, then LLM triage at confidence ≥ 0.7, then a human); calling agents in order; passing facts between them; running conditional steps; syncing Guardrail decisions; running executors once; writing the run summary; raising alerts.
Model	Llama 3.1 8B Instruct (Hugging Face). Its LLM tasks are short: classify free text into a route, and summarise a run in two sentences. A small, fast model is enough and keeps cost low. The control flow itself is deterministic code, so a run is repeatable and can resume after a restart.
Tools	`route catalog + keyword rules` `LLM triage` `agent adapters` `PostgreSQL event bus` `GitHub REST API (read)` `executors` `alerts`

Diagnostic / RCA

zrharounesprit

Objective	Explain why a build failed (objective O1).
Responsibilities	Clean the log, extract the error context, retrieve relevant documentation, and rank up to three root causes, each with a confidence and the source it relies on.
Model	DeepSeek-R1-Distill-Qwen-7B (Hugging Face), a reasoning model. Root-cause analysis means weighing several error lines against documents step by step, where a reasoning model is stronger than a plain instruction model of the same size. Its reasoning is stripped before parsing.
Tools	`log cleaner and error extractor` `knowledge-base search (pgvector)` `few-shot examples` `citation validator` `rule-based ranker (fallback)`

Dependency Resolver

Lyna

Objective	Unblock infrastructure deploys that fail on circular dependencies (O3).
Responsibilities	Build the resource graph of the OpenTofu/Terraform code, find the cycles, classify each one and explain how to break it.
Model	Llama 3.1 8B Instruct. The graph analysis is done by tools; the model only classifies the cycle and writes the explanation, a short and well-scoped task.
Tools	`tofu graph` `networkx cycle detection` `DOT parser (pydot)` `static .tf parser (fallback)` `keyword classifier (fallback)`

Merge Assistant

Muaad

Objective	Detect merge conflicts before a person merges, and propose a resolution (O3).
Responsibilities	Try the merge in a scratch copy, list the conflicting files and hunks, describe the conflict and propose a resolution. It never merges.
Model	Llama 3.1 8B Instruct: explaining a handful of conflicting lines does not need a larger model.
Tools	`GitPython` `scratch clone` `trial merge (--no-commit)` `conflict extractor` `rule-based description (fallback)`

Code Reviewer

hendch

Objective	Review pull requests for ML and MLOps mistakes and propose safe fixes (O2).
Responsibilities	Read the changes, lint findings and failing tests; report findings by severity; produce a minimal patch limited to safe lint fixes, proposed as a draft pull request.
Model	Grok 4.20, non-reasoning (xAI). The reviewer works by calling tools to read the code. The Hugging Face coder models we tried did not support tool calling through the router, and Grok handles it reliably.
Tools	`list_changed_files` `get_diff` `get_lint_findings` `get_test_failures` `get_rules` `submit_review` `ruff` `read-only git clone` `patch fixer and validator`

Drift Detection

omar

Objective	Notice when a production model's data or accuracy drifts, explain why, and recommend retraining, promotion or rollback (O3).
Responsibilities	Score each new production batch with statistical tests, investigate significant drift, and propose a report plus, when needed, a model change.
Model	Grok 4.20, reasoning (xAI). The investigation chains several tool calls and needs to separate data drift from concept drift and from a pipeline bug, which is multi-step reasoning. Statistics decide *whether* something drifted; the model explains *why*.
Tools	`get_drift_report` `compare_distributions` `segment_breakdown` `feature_importance` `drift_history` `recent_pipeline_events` `performance_history` `model_info` `model registry` `rule-based investigator (fallback)`

Guardrail

Rayen

Objective	Make sure no agent makes a risky change on its own (O4), and record every decision (O5).
Responsibilities	Validate every proposal; compute its real risk from evidence; apply the policy; ask an LLM judge for a second opinion; open approvals and enforce who may approve; expire stale approvals; write the hash-chained audit log.
Model	Qwen 2.5 7B Instruct (Hugging Face, via Featherless). It belongs to a different model family from the agents it checks, so it is less likely to share their blind spots, and it is reliable at strict JSON. The judge can only make a decision stricter, never looser.
Tools	`risk scanner` `Open Policy Agent (Rego)` `LLM judge` `approvals store` `hash-chained audit log` `approvals API`

How the Guardrail decides

1. **Risk from evidence.** Every agent labels its own proposal "low". The Guardrail scans what the proposal actually touches and can only raise the risk. Production, destructive operations and IAM/secrets raise it to high; billable infrastructure and state files raise it to medium; output produced by a fallback raises it one level.
2. **Policy.** Rego rules evaluated by OPA, first match wins. An agent proposing an action it is not allowed to is rejected. High risk in production needs two different people. Other high-risk changes need one. Advice is approved automatically. Low risk outside production is approved automatically. Anything unmatched is held (fail closed).
3. **LLM judge.** Turns an automatic approval into a hold when it judges the change risky. If the model is unreachable, the policy decision stands.

How information flows

- **One id per incident.** Each run's id is the correlation id on every event, approval, audit row and LangSmith trace, so the whole story of an incident can be followed.
- **Facts between agents.** Agents leave facts for later steps. The RCA diagnosis of a failing PR is handed to the Code Reviewer, so the review can say why the tests broke. If Drift Detection classifies drift as a pipeline bug, it triggers an RCA step.
- **Proposals as events.** Agents never act. Their output becomes events on the bus. The Guardrail answers with approval events, and the orchestrator picks up decisions and

runs the executors.

- **Reliable delivery.** Events are written in the same transaction as their log entry. Each consumer keeps a receipt per event. Failed events are retried with a growing delay, and marked dead after five attempts.

Routes: which agents handle which trigger

TRIGGER	AGENTS, IN ORDER
Failed CI build	Diagnostic / RCA
Failed deployment	Dependency Resolver → Diagnostic / RCA
Merge request	Merge Assistant
Pull request	Diagnostic / RCA *if its tests failed* → Code Reviewer
Drift check	Drift Detection → Diagnostic / RCA *if the drift is a pipeline bug*

2b

Monitoring the agents with LangSmith

A multi-agent system is hard to debug from logs: when a run gives a poor answer, you need to know which agent produced it, what that agent's model was asked, what it answered and how long it took. We integrated **LangSmith**, LangChain's tracing and evaluation platform, for this. Our agents do not use LangChain; LangSmith's SDK traces any Python code.

What is traced

Each orchestrator run becomes one trace, with a span per agent step and each model call nested inside its agent:

```
run: Failed deploy: IAM cycle              tags: orchestrator · metadata: run_id, route
├── Dependency Resolver                    status, summary, proposals
│   └── llm:dependency_resolver            prompt, answer, tokens, latency
├── Diagnostic / RCA
│   └── llm:diagnostic_rca                 DeepSeek-R1 call
└── (Code Reviewer, for pull requests)     runs in its own process and joins this trace
    └── llm:code_reviewer × N              one span per tool-calling turn
```

```
guardrail: dependency_resolver/propose_fix  risk, decision, rule, LLM verdict
└── llm:guardrail
execute and summarise                       executor results, LLM summary
└── llm:orchestrator
```

All traces of a run share a `thread_id` equal to the run id, so LangSmith groups the agents, the Guardrail's decisions (which happen asynchronously) and the execution after approval into a single thread.

How it is built

- `shared/tracing.py` holds the integration. Tracing is on only when `LANGSMITH_TRACING=true` and an API key is set. Otherwise every helper does nothing, so the app runs the same without an account.
- Every model call in the system goes through one client in `shared/llm.py`, which is wrapped with LangSmith's `wrap_openai`. The Code Reviewer and Drift Detection agents have their own clients, which are wrapped the same way.
- The Code Reviewer runs as a separate process. The orchestrator passes it the parent trace's headers, so its spans attach to the right run.
- The dashboard's Settings page shows whether tracing is on and which LangSmith project it writes to.

What it gives us

QUESTION	WHERE THE ANSWER COMES FROM
Which agent is slowest, or costs the most tokens?	Latency and token counts per span, filtered by agent tag
Why did a run give a wrong diagnosis?	The exact prompt, retrieved documents and answer of that model call
How often does each model fail and fall back to rules?	Error rate per agent tag
Why was this proposal held for approval?	The Guardrail span: risk, matched rule and LLM verdict

Two kinds of monitoring. The hash-chained audit log answers "what was decided, by whom, and has anyone altered the record?". LangSmith answers "how did each agent and model

behave?". The first is for accountability, the second for debugging and improving the agents.

Status: integrated and covered by unit tests; tested with tracing switched on, offline. Live traces start as soon as the team's LangSmith API key is added to `.env`.

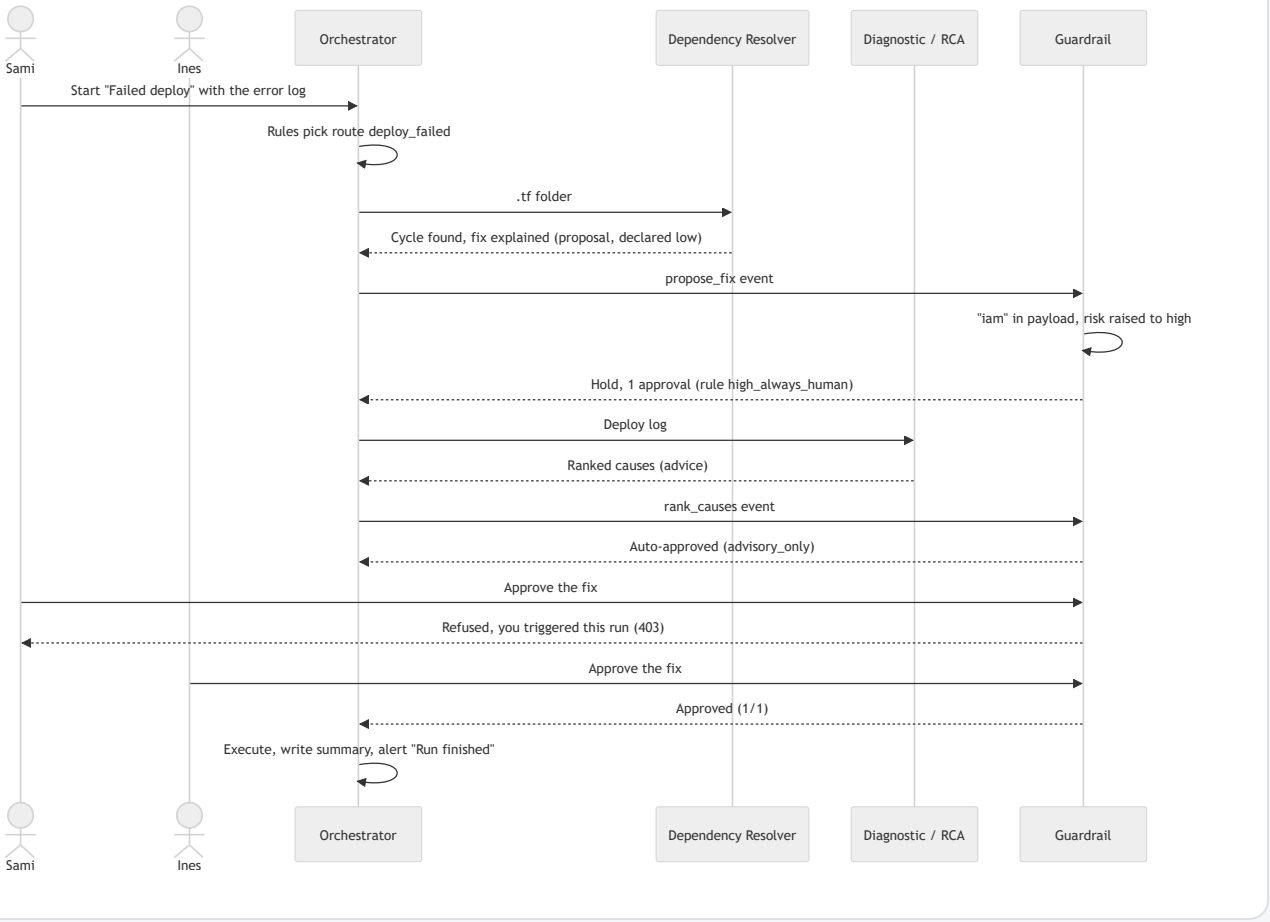
3

Prototype demonstration

The prototype runs locally with one command (`python -m app`). That starts the database, the policy engine, the agents, the GitHub watcher and the dashboard at `localhost:8080`. The dashboard has 14 pages: Home, Activity, Runs, run detail, Approvals, Triage, Alerts, four Analytics views, Agent registry, Audit log, Repositories and Settings.

Use case: a deploy fails on Friday evening

User: Sami, a platform engineer. **Input:** `tofu apply` fails with "Cycle: aws_iam_role.checkout, aws_eks_cluster.checkout, aws_iam_role_policy.checkout".



1. **Input.** Sami starts the scenario from the dashboard with the error log. Failed CI runs on a connected repository start runs the same way, automatically.
2. **Orchestrator.** Rules recognise a deployment failure and choose the Dependency Resolver, then the RCA agent.
3. **Dependency Resolver.** Finds the three-resource cycle and explains how to break it.
4. **Guardrail.** The fix touches IAM, a permissions change, so the risk goes from low to high and the fix is held for a human. The RCA ranking is advice, so it is approved at once.
5. **Approval.** Sami's own approval is refused. Ines approves from the Approvals page.
6. **Output.** The run completes with a summary written by the orchestrator's model. The run page shows the steps, the proposals with their risk and decision, and a timeline. The Audit page shows every decision, with the hash chain intact.

Other scenarios in the demo

SCENARIO	WHAT THE AGENTS PRODUCED, WITH THE REAL MODELS
Review a pull request (scaler fitted before the train/test split)	Code Reviewer: 6 findings, 4 of them high severity, including the data leakage, plus a safe lint patch proposed as a draft PR (dry run)
Failed CI build (pytest cannot reach the database)	RCA: top cause is the missing `DATABASE_URL` in CI, citing the matching knowledge-base document
Merge two branches (both change `TIMEOUT`)	Merge Assistant: conflict in `config.py` detected and a resolution proposed
Free-text request the model is unsure about	LLM triage confidence below 0.7, so the run waited on the Triage page for a person to pick the route

Data science and technical approach

Data

DATA	USED BY	DETAILS
Knowledge base of CI/CD failure documentation	RCA	Embedded with BAAI bge-small-en-v1.5 (384 dimensions) and stored in PostgreSQL with pgvector. The development seed has 5 documents; production use adds team runbooks.
Failed build logs	RCA	A sample failed pytest log for the demo; real GitHub Actions logs once a repository is connected. Secrets are removed before storage or any model call.
Few-shot examples	RCA	Worked log → ranked-causes pairs that show the model the expected output.
OpenTofu configurations	Dependency Resolver	Demo infrastructure with an IAM role / policy / EKS cycle, plus test configurations.
Pull requests	Code Reviewer	A demo PR with data leakage; real PRs cloned read-only once connected. Ruff lint results and failing tests feed the review.
UCI Bank Marketing (45,211 rows)	Drift Detection	Split into 45% training, 20% reference and a production pool that is replayed batch by batch, with simulated covariate drift, concept drift and pipeline bugs.
Decisions and outcomes	Analytics	Runs, approvals and executions stored in PostgreSQL feed the Analytics pages: time to decision, findings by severity, reviewer time saved.

Machine learning and AI components

COMPONENT	TECHNIQUE	OBJECTIVE IT SERVES
Retrieval-augmented diagnosis	Sentence embeddings, cosine search in pgvector (distance cutoff 0.3, calibrated on the seed), one query per error line, few-shot prompting of a reasoning LLM	O1
Answer validation	Structured JSON output, citations checked against retrieved documents, low-confidence causes dropped, retry when the model copies an example	O1, O5
Tool-using review agent	LLM function calling over a read-only toolbox, with forced submission at the end	O2
Graph analysis	Cycle detection on the resource dependency graph (networkx)	O3
Drift detection	Kolmogorov–Smirnov and PSI for numeric features; chi-square, Jensen–Shannon distance and PSI for categorical ones; ROC-AUC and F1 when labels arrive	O3
Production model under watch	HistGradientBoostingClassifier with permutation feature importance; versioned registry for promotion and rollback	O3
Intent classification	LLM triage with a confidence threshold, falling back to a human	O3, O4
LLM as a judge	A separate model family reviews proposals and can only escalate	O4
Observability	LangSmith tracing of every agent step and model call	O5

Main technical choices

- **Deterministic orchestrator, LLMs inside the agents.** Routing and state live in code and in the database, so runs are repeatable, resumable after a crash and easy to test. LLMs are used where judgement adds value.
- **Different models for different jobs.** A reasoning model for root causes, a tool-calling model for code review, a small fast model for short tasks, and a different family for the

judge.

- **Policy as code.** The approval rules are written in Rego and evaluated by Open Policy Agent, so they can be read, tested (20 policy tests) and changed without touching the agents.
- **PostgreSQL for everything.** Records, the event bus (LISTEN/NOTIFY), the vector store and the drift data share one database, set up by six numbered migrations. That means one system to run and back up.
- **Graceful degradation.** Each agent has a rule-based fallback, failing models are skipped for 30 seconds, and the Guardrail treats fallback output as less trustworthy.

5

Validation and feedback

170	152	18	20/20
automated tests	passing	skipped (drift data missing)	policy tests

Automated tests

AREA	TESTS	WHAT THEY CHECK
Guardrail	48	Risk scoring, every policy rule, approvals (self-approval, double votes, expiry), malformed events, audit chain
Drift Detection	30	Statistical tests, PostgreSQL store, investigator; 18 skipped until the data package is committed
Code Reviewer	27	Tools, safety limits, patch validation, LLM loop, end-to-end review
Orchestrator and GitHub	26	Routing, triage, conditional steps, approvals sync, execution, recovery, GitHub alerts and sign-in
Shared platform	14	Migrations, event bus delivery and retries, knowledge base
Dependency Resolver	12	Static parser and classifier fallbacks

AREA	TESTS	WHAT THEY CHECK
LLM client and redaction	10	Per-agent model settings, reasoning output, cooldown, secret masking
LangSmith tracing	3	No-op when off, client wrapped when on
OPA policy (Rego)	20	Each rule and its order, run with `opa test`

End-to-end checks with the real models

CHECK	RESULT	OBJECTIVE
RCA on the failed pytest log	met Correct top cause, with a real citation	O1
Code review of the leaky PR	met Data leakage found among 6 findings; patch produced	O2
IAM cycle and merge conflict	met Both detected and explained	O3
Risk escalation	met Agent's "low" raised to "high" and held	O4
Approval rules	met Self-approval refused (403); second vote refused (409)	O4
Audit integrity	met Hash chain verified on the Audit page	O5
Unsure triage	met Sent to a human instead of guessing	O4, O6
Model outage	met Rule-based fallbacks used and the run marked degraded (unit-tested)	O6
Drift scenario	not yet Agent cannot start without its data package	O3
Live GitHub repository	pending Built and unit-tested; not yet run against a real account	O1, O2

Benchmark: 64 cases with known answers

To measure the agents rather than demo them, we built a separate benchmark repository (`opsagents-bench`). It contains a small churn-model project with CI, and 64 cases, each a change with a known right answer:

- 8 pushes that break CI; the harness runs the real tests to get a real failing log;
- 8 pull requests: 6 with a planted ML mistake and 2 clean ones;
- 6 OpenTofu configurations;
- 5 branch pairs to merge;
- 15 incident reports to route;
- 22 proposals for the Guardrail.

The expected answers were written before running the agents. The same cases were also run with every model unreachable, to measure what the LLMs add over the rule-based fallbacks.

AGENT	METRIC	LIVE MODELS	RULES ONLY
Orchestrator	Correct route	**13/15**	8/15
	Requests only an LLM can route	7/7	0/7
Guardrail	Correct decision	**20/22**	17/22
	Unsafe auto-approvals (target 0)	**1**	4
	Risky changes with no keyword caught	3/4	0/4
	Benign changes wrongly blocked	0	0
Diagnostic / RCA	Top cause correct	**6/8**	4/8
Code Reviewer	Planted issues found	**6/6**	0/6
	False alarms on clean PRs	0	n/a
Dependency Resolver	Cycles detected, exact resources	6/6	6/6
Merge Assistant	Conflicts detected	5/5	5/5

What it showed.

- The LLMs add the most to routing, to the Guardrail's judgement and to code review. Infrastructure and merge detection rely on graph analysis and git, so they score the same either way.
- The failures are concrete and fixable:
  - a prompt injection in an incident report steered the router;
  - a patch that commented out the authentication middleware was auto-approved, the one unsafe decision;
  - a production model rollback that its agent labelled low needed one approver instead of two;
  - for an accuracy regression, the RCA agent copied an unrelated knowledge-base document instead of diagnosing the log.
- Answers vary between runs, so 64 cases is a first measurement, not a final score.

Limitations

- **No user accounts yet.** Who is acting is chosen in the dashboard, so the approval rules apply to names, not verified identities.
- **Nothing is written back to GitHub.** Approved patches are recorded as dry runs; draft PRs and review comments are not published yet.
- **The RCA knowledge base is small.** Five seed documents prove the pipeline but limit the quality of citations on unfamiliar failures.
- **A small evaluation set.** The 64-case benchmark is synthetic and small; the agents have not yet been measured on real incidents from the team's repositories.
- **Partial coverage of GitHub.** The app polls every 60 seconds instead of receiving webhooks, and merge and drift checks are not triggered from GitHub.
- **Local deployment only.** Free model tiers have rate limits; outages are absorbed by the fallbacks.

Difficulties encountered

- **Integrating six codebases.** The agents were written separately with different databases, model clients and environments. They now share one environment, one PostgreSQL database and one model client.
- **Model quirks.** DeepSeek-R1 returns nothing in JSON mode, so JSON is extracted from its free-text answer instead. Hugging Face coder models did not support tool calling, so the reviewer moved to Grok. Qwen 2.5 is only served through a specific provider.

- **Calibration.** The retrieval cutoff of 0.6 let irrelevant documents through; 0.3 was chosen on the seed set.
- **Missing code.** A broad `data/` ignore rule kept the drift agent's own package out of the repository.

6

Next steps

PRIORITY	TASK	OWNER
P1	Commit the drift data package and train the first model, so the drift scenario runs end to end	omar
P1	Connect the team repository and validate the live flow: failing push → alert → RCA; new PR → review	Rayen
P1	Add the LangSmith API key and collect real traces: latency, tokens and error rate per agent	Rayen
P1	Install OpenTofu so the Dependency Resolver uses the real dependency graph	Lyna
P1	Fix the benchmark's findings: treat model rollbacks and disabled authentication as high risk in the Guardrail; harden the router against instructions inside incident text	Rayen
P2	Grow the benchmark (more cases per agent, repeated runs to measure variance) and run it on every change, with LangSmith evaluations	zrharounesprit, hendch
P2	Grow the RCA knowledge base with real runbooks and past incidents	zrharounesprit
P2	Sign people in with GitHub so approvals are tied to real identities	Rayen
P2	Publish approved patches as draft PRs and post review findings as PR comments	hendch
P2	Run the Merge Assistant on real PR branches	Muaad

PRIORITY	TASK	OWNER
P3	Receive GitHub webhooks and deploy the platform	Team

The first phase proved the architecture: specialist agents that propose, a guardrail that decides, and a person who approves what matters. The next phase is about measuring and improving the agents' accuracy on real data, using the evaluation sets and the LangSmith traces.

Prepared for the Advanced Data Science prototype validation. Figures come from the code on branch `orchestrator` and test runs on 20 October 2026.